

Héritage et polymorphisme

Présentation : Tom Niget

Mail de contact : polytech@niget.fr

Site et ressources : niget.fr

Cours réalisé grâce notamment aux ressources de :
Julien Deantoni, Benjamin Werner, Joseph Albahari

Passage de paramètres

- Un paramètre se comporte comme une variable locale, et reçoit la valeur passée lors de l'appel.
- Cette valeur est systématiquement copiée : la modifier ne modifie pas la valeur originale
- Comment donc passer une valeur sans la copier, ou en pouvant la modifier ?

```
void display(Resistor resistor); // copie  
void display(Resistor* resistor); // pointeur, éventuellement nul  
void display(Resistor& resistor); // référence, sans copie
```

```
void display(const Resistor* resistor);  
void display(const Resistor& resistor);
```

Cas où on veut éviter une copie : objet qui possède une ressource, objet qui occupe beaucoup d'espace mémoire, ...

Bon à savoir : les appellations « passage par adresse » et « passage indirect » regroupent les passages par référence et par pointeur

Mise en commun d'éléments

- **Composants passifs élémentaires** : résistance, condensateur, inductance
- Dans un circuit, caractérisés par un **nom** et une **impédance**

```
class Component {  
public:  
    const std::string& name() const { return name_; }  
    std::complex<double> impedance(double frequency_hz) const { ... }  
    Component(std::string name) : name_(name) {}  
  
private:  
    std::string name_;  
};
```

Mise en commun d'éléments

```
class Component {
public:
    virtual ~Component() = default;
    const std::string& name() const { return name_; }
    virtual std::complex<double> impedance(double frequency_hz) const = 0;
```

```
protected:
    Component(std::string name) : name_(name) {}
```

```
private:
    std::string name_;
};
```

virtual = la méthode peut être définie par une classe fille (ou sous-classe). Avec « = 0; », elle doit être définie par une classe fille.

protected = la méthode ne peut être appelée que par une classe fille.



```
class Resistor : public Component {
public:
    Resistor(std::string name, double resistance_ohm)
        : Component(name), resistance_ohm_(resistance_ohm) {}
```

```
    std::complex<double> impedance(double frequency_hz) const override {
        return {resistance_ohm_, 0.0};
    }
```

```
private:
    double resistance_ohm_;
};
```

override précise qu'on est en train de (re)définir une méthode virtuelle de la classe mère. Le compilateur s'assure que la signature correspond.

Polymorphisme

Polymorphisme (en gros) : le fait qu'une même opération puisse s'appliquer à des choses de types différents.

Exemple : en mathématiques, « + » existe à la fois sur les nombres, sur les vecteurs, et sur les matrices.

En programmation objet, on réalise ça via le sous-typage : si B est une classe fille de A (« B hérite de A »), alors partout où on attend un A, on peut donner un B (car B est un « cas particulier » de A).

```
void display_at(const Component& component, double frequency_hz) {  
    ... component.name() ... component.impedance(frequency_hz) ...  
}
```

display_at prend une référence vers un **Component**, on peut lui passer un objet de type **Resistor** car c'est un sous-type de **Component**. On peut utiliser tous les membres exposés par **Component**.

Disposition mémoire et slicing

Stockage en mémoire de la classe Resistor du TD 1 :

```
class Resistor {  
public:  
    Resistor(std::string name, double resistance_ohm);  
  
    const std::string& name() const;  
    double resistance_ohm() const;  
    double voltage(double current_ampere) const;  
    std::string description() const;  
  
private:  
    std::string name_;  
    double resistance_ohm_;  
};
```

Position		
0	name_	buffer
4		
8		size
12		
16		capacity
20		
24	resistance_ohm_	
28		

Disposition mémoire et slicing

```
class Component {
public:
    virtual ~Component() = default;
    const std::string& name() const { return name_; }
    virtual std::complex<double> impedance(double frequency_hz) const = 0;

    ...

private:
    std::string name_;
};

class Resistor : public Component {
public:
    ...

    std::complex<double> impedance(double frequency_hz) const override {
        return {resistance_ohm_, 0.0};
    }

private:
    double resistance_ohm_;
};
```

Stockage en mémoire de la classe Resistor :

Component

Position	
0	vptr
4	
8	name_ buffer
12	
16	
20	
24	capacity
28	
32	resistance_ohm_
36	

(vptr est ici un pointeur vers un objet spécial qui contient les méthodes virtuelles. Il n'est présent que s'il y en a au moins une.)

Disposition mémoire et slicing

```
void display_at(Component component, double frequency_hz) {
    ...
}
```

```
Resistor r("R1", 1000.0);
display_at(r, 1000.0);
```

Component

Stocker une valeur Resistor dans une variable de type Component va « **découper** » (slice) l'objet et ne garder que les informations de Component.

Lors de l'affichage, le code essaiera de lire le contenu de resistance_ohm_ qui n'existe plus → illégal !

On doit donc **impérativement faire un passage indirect** (pointeur ou référence).

Stockage en mémoire de la classe Resistor :

Position		
0	vptr	
4		
8	name_	buffer
12		
16		size
20		
24		capacity
28		

Destruction

- Rappel : le destructeur est la méthode appelée lorsque l'objet cesse d'exister
- C'est là qu'on va par exemple libérer des ressources gérées par l'objet
- Et s'il y a de l'héritage ?

```
class Component {  
public:  
    virtual ~Component() = default;  
    ...  
};
```

```
class Resistor : public Component {  
public:  
    // automatiquement : ~Resistor() appelle ~Component()  
    ...  
};
```

Le destructeur de Component doit être marqué comme virtuel pour que les classes filles puissent correctement l'appeler pour détruire les champs de la classe mère.

« = `default` » signifie qu'on laisse le destructeur par défaut (qui détruit automatiquement tous les champs).

Nombres complexes

- La librairie standard fournit le type `std::complex<T>` (où T est soit `float` soit `double`)

```
std::complex<double> z1(1.0, 2.0);  
std::complex<double> z2(3.0, 4.0);  
std::complex<double> sum = z1 + z2;  
std::complex<double> product = z1 * z2;  
double magnitude = std::abs(z1);  
double phase = std::arg(z1);
```

```
std::complex<double> coord_polar(double magnitude, double phase) {  
    return {magnitude * std::cos(phase), magnitude * std::sin(phase)};  
}
```